Revisão de estrutura de árvores

(Atividade individual remota para o dia 28/09)

Nome: Silviane M. A. de Souza Ribeiro
Disciplina: Estrutura de Dados II **Professora:** Profa. Kadidja Valéria
Modalidade: individual, remota e assíncrona **Tempo estimado:** 1 hora e 45 minutos

Etapa 1 — Revisão bibliográfica (35 minutos)

A árvore é uma estrutura de dados hierárquica formada por nós e conexões. A raiz é o nó que não possui pai, enquanto as folhas são nós sem filhos. O nível de um nó corresponde ao comprimento do caminho desde a raiz até ele, somado de 1; a altura da árvore é o maior nível existente. A principal diferença em relação a estruturas lineares, como listas, é que uma árvore permite representar relações hierárquicas e dividir a busca em diferentes caminhos.

Árvore geral: não estabelece limite fixo para a quantidade de filhos de cada nó. É adequada para representar hierarquias nas quais um elemento pode possuir vários descendentes, como estruturas organizacionais. A busca depende da forma de organização adotada e pode exigir a visita de muitos nós.

Árvore binária: cada nó possui no máximo dois filhos, identificados como esquerdo e direito. Os filhos podem ser vazios. A estrutura é útil como base para outras árvores, como as árvores binárias de busca e heaps.

Árvore binária de busca (ABB/BST): mantém os valores menores na subárvore esquerda e os maiores na subárvore direita. A busca compara o valor procurado com o nó atual e decide para qual lado seguir. Quando a árvore permanece aproximadamente balanceada, a busca pode ser O(log n); quando fica degenerada, pode chegar a O(n). O percurso em-ordem produz os valores em ordem crescente.

AVL: é uma árvore binária de busca autoequilibrada. Para cada nó, as alturas das subárvores esquerda e direita diferem no máximo em 1. Depois de inserções ou remoções, podem ser necessárias rotações para restaurar o equilíbrio. Essa manutenção garante operações de busca, inserção e remoção em tempo logarítmico no pior caso.

Rubro-negra: é uma árvore de busca binária que utiliza informações de cor e regras de balanceamento para limitar sua altura. Durante alterações podem ocorrer rotações e recolorações. Essas operações preservam a ordem da BST e o balanceamento necessário para manter as operações em O(log n) no pior caso.

Heap: é uma árvore binária completa com ordenação parcial entre pais e filhos. No max-heap, o pai é maior ou igual aos filhos e o maior elemento fica na raiz; no min-heap ocorre o contrário. É especialmente útil para filas de prioridade e para operações de remoção do maior ou menor elemento.

Árvore B: é uma árvore balanceada de múltiplos caminhos, na qual um nó pode armazenar várias chaves e possuir vários filhos. Os nós são mantidos dentro de limites de ocupação e as folhas ficam no mesmo nível. Quando um nó fica cheio durante uma inserção, pode ocorrer divisão do nó e promoção de uma chave. É adequada para armazenamento e busca em estruturas externas, como arquivos e páginas de disco.

Árvore B+: é uma variação da árvore B em que os registros ficam nas folhas, enquanto os nós internos armazenam chaves usadas para orientar a busca. As folhas normalmente são ligadas entre si, facilitando a leitura sequencial e consultas por intervalo. Na inserção, uma folha cheia pode ser dividida, podendo provocar divisões nos níveis superiores.

Trie: é uma estrutura voltada principalmente para chaves formadas por caracteres. Os caminhos representam sequências de caracteres e palavras que compartilham um prefixo aproveitam os mesmos nós iniciais. É útil para dicionários, autocompletar, busca por prefixos e outras aplicações envolvendo strings.

Etapa 2 — Quadro comparativo (35 minutos)

Preencha todas as linhas pertinentes ao conteúdo da disciplina. Acrescente árvore geral, árvore binária, heap e trie quando forem abordadas nas referências ou aulas. A coluna de referência deve apontar a fonte específica usada para cada linha.

Estrutura	Organização dos dados	Regra ou propriedade	Operação ou ajuste	Aplicação	Referência
ABB	Menores à esquerda e maiores à direita.	Para cada nó: esquerda < nó < direita, evitando duplicatas conforme a convenção adotada.	Busca segue esquerda/direita; não há balanceamento automático.	Busca ordenada e representação de conjuntos / mapas.	Material da disciplina, cap. 6 — árvores binárias de busca.
AVL	BST com controle de altura das subárvores.	Diferença de alturas entre subárvores de qualquer nó é no máximo 1.	Rotações após inserção/remoção para restaurar equilíbrio.	Estruturas que precisam de buscas ordenadas com desempenho previsível.	OpenDSA, Balanced Trees; Princeton AVLTreeST.
Rubro-negra	BST com informação de cor nos nós/links.	Regras de cores e igual número de links pretos nos caminhos mantêm a altura controlada.	Rotações e recolorações após inserção/remoção.	Mapas e conjuntos ordenados; ex.: TreeMap do Java.	Princeton, Balanced Search Trees; Oracle TreeMap.
B	Cada nó pode armazenar várias chaves e vários ponteiros.	Árvore balanceada; folhas no mesmo nível e nós dentro de limites de ocupação.	Divisão de nós cheios e r edistribuição/combinação em remoções, conforme o caso.	Arquivos, índices e armazenamento em disco	OpenDSA, B-Trees.
B+	Chaves orientadoras nos nós internos; registros nas folhas, normalmente ligadas.	Folhas no mesmo nível; folhas mantidas com ocupação mínima.	Divisão de folhas/nós e promoção de chaves; folhas encadeadas favorecem intervalos.	Índices de bancos de dados e consultas por intervalo.	OpenDSA, B-Trees/B+ Trees.

Etapa 3 — Identificação por analogias (35 minutos)

1. Uma estante de números é reorganizada por rotações quando um lado fica alto demais em relação ao outro.

Estrutura: **AVL**.

Justificativa técnica: A analogia corresponde a uma AVL porque essa estrutura é uma ABB que controla a diferença de altura entre as subárvores esquerda e direita. Quando uma inserção ou remoção faz o fator de equilíbrio sair do limite permitido, são usadas rotações para restaurar o equilíbrio.

Limite da analogia: Limite da analogia: uma estante real não possui regras matemáticas de altura nem realiza rotações automaticamente. A comparação representa apenas a ideia de reorganizar a estrutura para evitar que um lado fique muito alto.

2. Um catálogo guarda várias chaves por página; quando uma página fica cheia, ela dividida.

Estrutura: árvore B.

Justificativa técnica: A situação corresponde a uma árvore B porque seus nós podem armazenar várias chaves e possuir vários filhos. Quando um nó fica cheio durante uma inserção, ele pode ser dividido, mantendo a árvore balanceada.

Limite da analogia: Limite da analogia: uma página física de catálogo não é literalmente um nó de árvore B. A analogia representa a ideia de agrupar várias chaves em um bloco e dividi-lo quando sua capacidade é excedida.

3. Uma fila mantém a tarefa de maior prioridade no topo para retirá-la primeiro. Estrutura: **heap, especificamente max-heap**.

Justificativa técnica: No max-heap, cada pai possui valor maior ou igual aos filhos, fazendo com que o maior elemento fique na raiz. Por isso a estrutura é adequada para implementar uma fila de prioridade.

Limite da analogia: Limite da analogia: um heap não mantém todos os elementos totalmente ordenados. Ele garante principalmente a relação entre cada pai e seus filhos; portanto, os demais elementos não ficam necessariamente em ordem global.

4. Um índice percorre letras sucessivas e compartilha o início das palavras de mesmo prefixo. Estrutura: **trie**.

Justificativa técnica: A trie organiza chaves de caracteres por seus prefixos. Palavras que começam com as mesmas letras compartilham os nós correspondentes a esse início, permitindo buscas e consultas por prefixo.

Limite da analogia: Limite da analogia: um índice convencional de palavras pode armazenar informações adicionais e usar outras estruturas. A trie é uma representação específica baseada nos caracteres da chave.

5. Uma estrutura usa cores, recolorações e rotações para manter controlada a altura dos caminhos de busca.

Estrutura: **árvore rubro-negra**.

Justificativa técnica: A árvore rubro-negra é uma BST que associa cores aos nós ou links e utiliza rotações e recolorações para preservar suas invariantes de balanceamento. Isso limita a altura e mantém operações de busca, inserção e remoção em O(log n) no pior caso.

Limite da analogia: Limite da analogia: as cores não representam categorias ou prioridades dos dados. Elas são informações auxiliares usadas pelo algoritmo para controlar a estrutura da árvore.

6. Um índice conduz às folhas que contêm os registros, ligadas entre si para facilitar consultas por intervalo.

Estrutura: **árvore B+**.
Justificativa técnica: Na B+, os nós internos funcionam principalmente como guias de busca, enquanto os registros ficam nas folhas. As folhas são normalmente ligadas, permitindo percorrer sequencialmente os registros em ordem e realizar consultas por intervalo com eficiência.

Limite da analogia: Limite da analogia: um índice de banco de dados não é necessariamente implementado como uma única B+ nem todas as consultas são de intervalo. A analogia destaca apenas uma aplicação típica.

7. Numa coleção de números, cada nó direciona valores menores para a esquerda e maiores para a direita.

Estrutura: **árvore binária de busca (ABB/BST)**.

Justificativa técnica: Essa é a propriedade central da ABB: para cada nó, os valores menores ficam na subárvore esquerda e os maiores na direita, permitindo decidir o caminho durante uma busca por comparações.

Limite da analogia: Limite da analogia: apenas essa regra não garante que a árvore permaneça balanceada. Uma ABB pode ficar degenerada, parecendo uma lista ligada, e nesse caso a busca pode chegar a O(n).

REFERÊNCIAS CONSULTADAS

• Material disponibilizado na disciplina Estrutura de Dados II, capítulo 6 — “Árvores, árvores binárias e árvores binárias de busca” e seção 6.4 — “Percurso em árvores”.

• Virginia Tech / OpenDSA. Balanced Trees — AVL Tree.
https://opendsa-server.cs.vt.edu/ODSA/Books/fu/comp502/fall-2020/Franklin_Fall_2020/html/BalancedTree.html •

Princeton University, Algorithms, 3.3 Balanced Search Trees — Red-black BSTs.

https://algs4.cs.princeton.edu/33balanced/ • Oracle Java SE 26 API — TreeMap.

https://docs.oracle.com/en/java/javase/26/docs/api/java.base/java/util/TreeMap.html

• Virginia Tech / OpenDSA. Heaps and Priority Queues.
https://opendsa-server.cs.vt.edu/ODSA/Books/dc/comp232/fall-2021-bbf8bb5f-a92f-4361-9af3-980c57968258/B/html/Heaps.ht

ml • Virginia Tech / OpenDSA. B-Trees / B+-Trees.

https://opendsa-server.cs.vt.edu/ODSA/Books/CS3S26notes/html/BTrees.html • Princeton University, Algorithms — Tries.

https://algs4.cs.princeton.edu/lectures/keynote/52Tries.pdf